

Practical Week-9

Task 1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
lenovo@Manya:~$ mkdir -p ~/OSSP/FileCopy
lenovo@Manya:~$ cd ~/OSSP/FileCopy
lenovo@Manya:~/OSSP/FileCopy$ pwd
/home/lenovo/OSSP/FileCopy
lenovo@Manya:~/OSSP/FileCopy$ echo "This is a sample file f
or Linux file copy system call demonstration." > input.txt
lenovo@Manya:~/OSSP/FileCopy$ cat input.txt
This is a sample file for Linux file copy system call demonstration.
lenovo@Manya:~/OSSP/FileCopy$ nano lowlevel_copy.c
lenovo@Manya:~/OSSP/FileCopy$ gcc lowlevel_copy.c -o lowlevel_copy
lenovo@Manya:~/OSSP/FileCopy$ ls
input.txt  lowlevel_copy  lowlevel_copy.c
lenovo@Manya:~/OSSP/FileCopy$ ./lowlevel_copy input.txt output.txt
Source file size: 69 bytes
File copied successfully using low-level system calls.
lenovo@Manya:~/OSSP/FileCopy$ cat output.txt
This is a sample file for Linux file copy system call demonstration.
lenovo@Manya:~/OSSP/FileCopy$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[])
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;
    off_t fileSize;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    // Open source file for reading
    source = open(argv[1], O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    // Get file size using lseek()
    fileSize = lseek(source, 0, SEEK_END);

    if (fileSize < 0)
    {
        perror("lseek error");
        close(source);
        return 1;
    }

    printf("Source file size: %ld bytes\n", (long)fileSize);

    // Move file pointer back to beginning
    lseek(source, 0, SEEK_SET);

    // Create destination file
    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```

if (destination < 0)
{
    perror("Error opening destination file");
    close(source);
    return 1;
}

// Copy data
while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
{
    bytesWritten = write(destination, buffer, bytesRead);
    if (bytesWritten != bytesRead)
    {
        perror("Write error");
        close(source);
        close(destination);
        return 1;
    }
}

if (bytesRead < 0)
{
    perror("Read error");
}

close(source);
close(destination);

printf("File copied successfully using low-level system calls.\n");

return 0;
}

```

Task 2: Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

```

lenovo@Manya :~/OSSP/FileCopy$ nano standard_copy.c
lenovo@Manya :~/OSSP/FileCopy$ gcc standard_copy.c -o standard_copy
lenovo@Manya :~/OSSP/FileCopy$ ./standard_copy input.txt output2.txt
File copied successfully using standard library functions.
lenovo@Manya :~/OSSP/FileCopy$ cat output2.txt
This is a sample file for Linux file copy system call demonstration.
lenovo@Manya :~/OSSP/FileCopy$ |

```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    FILE *source, *destination;
    char buffer[1024];
    size_t bytesRead;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    source = fopen(argv[1], "rb");
    if (source == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen(argv[2], "wb");
    if (destination == NULL)
    {
        perror("Error opening destination file");
        fclose(source);
        return 1;
    }

    while ((bytesRead = fread(buffer, 1, sizeof(buffer), source)) > 0)
    {
        fwrite(buffer, 1, bytesRead, destination);
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library functions.\n");

    return 0;
}

```

```

lenovo@Manya:~$ cd ~/OSSP
lenovo@Manya:~/OSSP$ mkdir -p Redirection
lenovo@Manya:~/OSSP$ cd Redirection
lenovo@Manya:~/OSSP/Redirection$ nano redirect.c
lenovo@Manya:~/OSSP/Redirection$ echo "Hello from input file." > input.txt
lenovo@Manya:~/OSSP/Redirection$ echo "This is Linux system call demonstration." >> input.txt
lenovo@Manya:~/OSSP/Redirection$ echo "dup2 is used for I/O redirection." >> input.txt
lenovo@Manya:~/OSSP/Redirection$ cat input.txt
Hello from input file.
This is Linux system call demonstration.
dup2 is used for I/O redirection.
lenovo@Manya:~/OSSP/Redirection$ gcc redirect.c -o redirect
lenovo@Manya:~/OSSP/Redirection$ ls
input.txt  redirect  redirect.c
lenovo@Manya:~/OSSP/Redirection$ ./redirect
Input and output redirection completed.
Check output.txt for the redirected output.
lenovo@Manya:~/OSSP/Redirection$ cat output.txt
Hello from input file.
This is Linux system call demonstration.
dup2 is used for I/O redirection.
lenovo@Manya:~/OSSP/Redirection$ |

```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int input_fd, output_fd;
    pid_t pid;

    // Open input file
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("Error opening input.txt");
        return 1;
    }

    // Open/create output file
    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);

    if (output_fd < 0)
    {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Redirect standard input to input.txt
        if (dup2(input_fd, STDIN_FILENO) < 0)
        {
            perror("dup2 input failed");
        }
    }
}

```

```
        perror("dup2 input failed");
        exit(1);
    }

    // Redirect standard output to output.txt
    if (dup2(output_fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 output failed");
        exit(1);
    }

    close(input_fd);
    close(output_fd);

    // Execute command
    execlp("cat", "cat", NULL);

    perror("execlp failed");
    exit(1);
}
else
{
    close(input_fd);
    close(output_fd);

    wait(NULL);

    printf("Input and output redirection completed.\n");
    printf("Check output.txt for the redirected output.\n");
}

return 0;
}
```